

## Resenha: Design by Contract

Ao mergulhar na leitura do trabalho de Bertrand Meyer sobre o Projeto por Contrato, percebemos que a engenharia de software ganha uma clareza quase filosófica ao adotar a metáfora dos contratos humanos para a construção de sistemas. Meyer argumenta que a confiabilidade de um programa, definida pela união entre correção e robustez, não nasce por acaso, mas sim de uma sucessão de decisões documentadas que estabelecem direitos e deveres mútuos entre os diferentes componentes de um software. É fascinante observar como essa ideia simples consegue atacar problemas complexos que atormentam programadores há décadas, como a gestão de exceções e a segurança no uso de herança. O autor nos convida a abandonar a ideia de que o software é um amontoado de códigos que simplesmente funcionam, propondo em vez disso uma visão onde cada chamada de rotina é uma transação formal protegida por um documento de contrato.

O coração dessa abordagem reside na relação entre o cliente, que solicita um serviço, e o fornecedor, que o executa. Em um contrato bem redigido, as obrigações de uma parte tornam-se os benefícios da outra. Para que o fornecedor faça o seu trabalho, o cliente deve garantir certas condições iniciais, chamadas de pré-condições. Em troca, o fornecedor compromete-se a entregar um resultado específico, as pós-condições, além de manter a integridade geral do objeto, o que Meyer define como invariantes de classe. Se o cliente falha em cumprir a pré-condição, o fornecedor não tem obrigação alguma de executar a tarefa. Essa distinção é crucial porque elimina a ambiguidade sobre quem é o responsável por verificar a validade dos dados, permitindo que o código seja mais limpo e focado na sua lógica principal.

Um dos ensinamentos mais provocativos do artigo é a crítica severa à chamada programação defensiva. Meyer explica que a prática de espalhar verificações redundantes por todo o sistema, onde tanto quem chama quanto quem é chamado conferem as mesmas condições, acaba sendo prejudicial à confiabilidade global. Esse excesso de zelo aumenta a complexidade da arquitetura, tornando os módulos difíceis de ler e manter. No modelo de contrato, a responsabilidade é atribuída a apenas um elemento do software, e essa clareza permite que as rotinas se concentrem em realizar bem uma tarefa precisa, em vez de tentarem tratar todos os casos anormais possíveis de forma desorganizada. O resultado é uma simplicidade arquitetônica que, ironicamente, produz sistemas muito mais robustos do que aqueles saturados de verificações defensivas.

A discussão sobre falhas e exceções também ganha um novo fôlego com as leis estabelecidas pelo autor. Meyer define falha como a incapacidade de uma rotina cumprir o seu contrato. Uma exceção, por sua vez, é qualquer evento anormal que ocorra durante a execução e que impeça o progresso normal de uma tarefa. A primeira lei do tratamento de exceções é categórica: uma rotina só pode terminar de duas formas, ou ela cumpre o contrato com sucesso, ou ela admite a falha e notifica o seu cliente. Não existe meio-termo ou mentira permitida. O autor critica duramente mecanismos de linguagens que permitem que uma rotina falhe silenciosamente, retornando ao chamador como se nada tivesse acontecido, o que ele compara a um construtor que não entrega a casa mas finge que o trabalho foi concluído.

Para lidar com esses momentos de crise, o artigo propõe um mecanismo disciplinado baseado em duas ações fundamentais: a recuperação e a tentativa. Quando algo dá errado,

a rotina pode tentar limpar o estado do sistema, restaurando os invariantes da classe, e então realizar uma nova tentativa usando a mesma estratégia ou uma abordagem alternativa. Se após as tentativas a meta ainda não for alcançada, a rotina deve entrar em pânico organizado, garantindo que o objeto esteja em um estado estável antes de sinalizar a falha para quem a chamou. Essa estrutura garante que o erro seja propagado de forma previsível até que alguém com o contexto adequado possa decidir como prosseguir, evitando que o sistema continue operando com valores sem sentido ou em estados inconsistentes.

A aplicação do Projeto por Contrato na herança e no polimorfismo revela-se uma ferramenta poderosa para evitar confusões técnicas. Meyer introduz a ideia de que a redefinição de um método em uma classe descendente é, na verdade, uma subcontratação. Assim como no mundo dos negócios, um subcontratado honesto deve respeitar o contrato original assinado pelo contratante principal. Isso se traduz na regra de que uma versão redefinida de uma rotina pode aceitar mais casos do que a original, enfraquecendo a pré-condição, e pode entregar um resultado mais preciso ou melhor, fortalecendo a pós-condição. O que ela jamais pode fazer é ser mais exigente com o cliente ou entregar menos do que o prometido originalmente, garantindo assim que o substituto possa ser usado em qualquer lugar onde o original era esperado sem quebrar a lógica do sistema.

Por fim, o ensinamento que encerra a reflexão de Meyer é o apelo à aceitação das funções parciais. Ele nos lembra que, na matemática e na programação, muitas operações não são aplicáveis a todos os casos possíveis, como tentar obter o topo de uma pilha vazia. Em vez de buscar o objetivo ilusório de construir componentes que funcionem sob qualquer circunstância imaginável, o Projeto por Contrato foca em fazer com que cada componente declare exatamente o que fará e o que não fará. Ao tornar essas fronteiras explícitas e documentadas, Meyer nos oferece não apenas uma técnica de codificação, mas uma filosofia de design que prioriza a honestidade técnica, a simplicidade e a construção de sistemas que podem ser verdadeiramente compreendidos e confiados por seres humanos.